

PROGRAMACIÓN AVANZADA

Sesión 23

Argumentos de consola



NOTAS DEL
PROFESOR

Índice

1	Objetivos de la sesión	2
2	Prerrequisitos	2
3	Arranque y preguntas de sondeo	3
4	main con parámetros	3
5	Demo en vivo: cuatro programas	4
5.1	hola_args.cpp: imprimir argc y argv	4
5.2	uso.cpp: verificar argc y mensaje de uso	5
5.3	suma.cpp: iterar sobre los argumentos	5
5.4	calc.cpp: el primer argumento como operador	5
6	Convertir argumentos a números	6
7	Actividad: el teclado numérico	6
8	Soluciones a los ejercicios	8
9	Errores frecuentes	10
10	Rúbrica de la actividad	11
11	Conexiones con temas posteriores	11
12	Materiales y recursos	12

Objetivos de la sesión

Al finalizar la sesión el alumno será capaz de:

1. Explicar la firma completa de `main` con `argc` y `argv`, y describir el valor de cada parámetro para una ejecución concreta.
2. Escribir programas que reciban argumentos de consola y los procesen con un ciclo sobre `argv`.
3. Verificar correctamente el número de argumentos e imprimir un mensaje de uso estándar en `cerr`.
4. Convertir argumentos de cadena a tipos numéricos con `atoi`, `atof` y `stoi`.
5. Resolver la actividad del teclado numérico usando `argc`, `argv` y una tabla de mapeo letra-tecla.

10 min Gancho histórico + preguntas de arranque
10 min `main` con parámetros: `argc` y `argv`
20 min Demo en vivo: cuatro programas (ver sección 4)
5 min Tabla de conversión a números
55 min Actividad: teclado numérico

La actividad es la parte más rica de la sesión. Los primeros 35 minutos se dedican a implementar; los 20 restantes a discutir soluciones y ver el diseño de la tabla de mapeo.

Prerrequisitos

- Ciclos `for` y arreglos (sesiones 7–8).
- Cadenas estilo C: `char*`, acceso por índice, terminador `'\0'` (sesiones 3 y 9).
- Tabla ASCII y operaciones con `char` (sesión 3).
- Compilación y ejecución básica desde terminal con `g++`.

Esta sesión no requiere archivos, structs ni memoria dinámica. Es conceptualmente ligera: los alumnos ya conocen arreglos y ya usan la terminal. El reto es que `argv` combina dos conceptos que dominan por separado (arreglos y cadenas) en un solo objeto que nunca habían visto así.

Señal de alerta: si algún alumno no recuerda que `char*` es un puntero a una cadena terminada en `'\0'`, tomar 3 minutos para mostrar que `argv[1][0]` es el primer carácter del primer argumento, antes de avanzar. El ejercicio 4 (contador) lo necesita directamente.

Arranque y preguntas de sondeo

1. “¿Cómo le pasan instrucciones a g++ cuando compilan?”

Respuesta esperada: escribiendo opciones en la línea de comandos (-o, -Wall, nombre del archivo). Seguir con: “eso significa que g++ recibe esos textos de alguna manera dentro de su `main`. Hoy van a aprender exactamente cómo.”

2. “¿Cuántos argumentos recibe g++ cuando ejecutan `g++ -O2 -o hola hola.cpp`?”

Respuesta esperada (varios intentos): la respuesta es 4. El nombre del programa (g++) se cuenta aparte y es `argv[0]`. Este pequeño “¿qué tal 4 o 5?” crea la tensión cognitiva perfecta para explicar `argc`.

3. “¿Qué diferencia hay entre `./programa` y `./programa hola mundo`?”

Respuesta esperada: en el segundo caso el programa recibe dos palabras extra. Seguir con: “¿cómo las recibe? ¿Por dónde entran? ¿Desde `cin`?” Respuesta esperada: no, `cin` lee del teclado *durante* la ejecución; los argumentos de consola se entregan *antes* de que el programa comience.

main con parámetros

Escribir en la pizarra o proyectar los dos `main` uno junto al otro:

```
int main()                                // el que siempre han usado
int main(int argc, char* argv[])          // la versión completa
```

Preguntar: “¿qué nuevo hay aquí?” Identificar `int argc` y `char* argv[]`. Explicar que `argv[]` es un arreglo de `char*`: cada elemento apunta a una cadena de texto terminada en `'\0'`.

Dibujar en la pizarra el diagrama de memoria para `./programa hola mundo` 42:

```
argv
+-----+ +-----+
| [0] |-->| "./programa\0" |
+-----+ +-----+
| [1] |-->| "hola\0"       |
+-----+ +-----+
| [2] |-->| "mundo\0"      |
+-----+ +-----+
| [3] |-->| "42\0"         |
+-----+ +-----+
argc = 4
```

Señalar que `argv[3]` es la cadena "42", no el número 42. Este punto se enfatiza antes de la demo de conversión.

¿Son lo mismo `char* argv[]` y `char** argv` ?

Sí, en el contexto de los parámetros de una función son equivalentes. Ambas formas se aceptan y es bueno que los alumnos lo sepan porque verán la segunda en código de C y en muchos libros.

¿Por qué `argc` incluye el nombre del programa?

Decisión de diseño de los primeros Unix: el programa puede necesitar saber cómo fue invocado. Un ejemplo clásico es `vi/vim`: si se invoca como `view` (un enlace simbólico) se abre en modo solo lectura. Otro: `argv[0]` en el mensaje de uso.

¿Qué hay en `argv[argc]` ?

El estándar garantiza que `argv[argc]` es `nullptr` (puntero nulo). Algunos programas aprovechan esto para iterar con `while (*argv++)` en lugar de un `for` con índice. Para este curso la forma con `for` e índice es más clara.

¿Puedo asignar `argv[i]` a un `string` ?

Sí. `string` tiene un constructor que acepta `const char*`: `string s = argv[1];` funciona. A partir de ahí se puede usar todas las funciones de `std::string` (longitud, búsqueda, etc.).

Demo en vivo: cuatro programas

• `hola_args.cpp`: imprimir `argc` y `argv`

1. Escribir el programa en vivo (es corto; no mostrar el código antes).
2. Compilar y ejecutar sin argumentos. Preguntar antes: "¿qué valor tiene `argc` ?" Respuesta: 1. Mostrar que solo imprime `argv[0]` .
3. Ejecutar con uno dos tres. Confirmar `argc = 4` y los cuatro valores en pantalla.
4. Ejecutar con "hola mundo" entre comillas. Mostrar que ahora `argc = 2` y `argv[1] = "hola mundo"` : el shell pasa la cadena completa como un solo argumento.
5. Preguntar: "¿qué índice tiene el último argumento en términos de `argc` ?" Respuesta: `argc-1` .

- **uso.cpp: verificar argc y mensaje de uso**

1. Mostrar la verificación `if (argc != 3)` y preguntar: “¿por qué 3 y no 2?”
Respuesta: el programa espera nombre y apellido, más `argv[0]`, son 3 cadenas en total.
2. Ejecutar con el número correcto de argumentos. Mostrar salida esperada.
3. Ejecutar con uno solo. Mostrar el mensaje de error en `cerr`. Preguntar: “¿por qué `cerr` y no `cout`?” Demostrar con `./uso Ana 2>/dev/null`: el mensaje de error desaparece; cualquier salida normal seguiría visible.
4. Señalar que `return 1` indica al shell que hubo error. Se puede demostrar con `echo $?` (Linux/Mac) o `echo %ERRORLEVEL%` (Windows) justo después.

- **suma.cpp: iterar sobre los argumentos**

1. Antes de mostrar el código, preguntar: “Si los argumentos son números, ¿cómo podríamos sumarlos?” Esperar propuestas. La respuesta natural es un `for` desde `i=1` hasta `argc-1` con `atoi`.
2. Mostrar el código. Compilar y ejecutar con varios números.
3. Probar con cero argumentos (`argc = 1`). El ciclo no itera y el resultado es 0: correcto.
4. Probar con un argumento no numérico (`./suma hola`). `atoi("hola")` devuelve 0. Preguntar: “¿es esto un buen comportamiento?” Introducir `stoi` como alternativa que lanza excepción.

- **calc.cpp: el primer argumento como operador**

1. Señalar el acceso `argv[2][0]` : el primer carácter del segundo argumento. Este es el único lugar del curso donde se usa la indexación doble sobre `argv`.
2. Compilar y ejecutar con varias operaciones. Demostrar que `./calc 10 / 4` da 2.5 (no 2, porque `atof` produce `double`).
3. Probar con un operador inválido. Mostrar el `return 1` en acción.
4. **Pregunta de profundidad:** “¿Qué pasa si el usuario escribe `./calc 10 * 4`?” Respuesta: el shell expande `*` como un glob antes de pasárselo al programa. Los alumnos verán algo inesperado. Solución: escapar el asterisco: `./calc 10 \* 4` o `./calc 10 "*" 4`. Esto abre una conversación interesante sobre qué hace el shell antes de invocar el programa.

Convertir argumentos a números

Presentar la tabla de funciones y enfatizar la simetría: `atoi / stoi` para enteros, `atof / stod` para flotantes. Las versiones con `a` son de C (reciben `char*`), las de `s` son de C++ (reciben `string`).

Regla práctica para este curso: usar `atoi / atof` cuando el dato viene directamente de `argv` (ya es `char*`); usar `stoi / stod` cuando el dato ya es un `string`.

Si alguien pregunta por `stoi` con cadenas inválidas, mencionar que lanza `std::invalid_argument` y que se captura con `try / catch`, tema que se ve en sesiones posteriores.

Actividad: el teclado numérico

Antes de la clase:

- Tener la imagen del teclado lista para proyectar.
- Preparar `materialInicial/sesion23/` con el archivo `actividad` y la imagen `teclado_numerico.jpg`.

Introducción (5 min):

Mostrar la imagen del teclado y preguntar a los alumnos cuántas pulsaciones requiere cada letra. Construir la tabla en vivo (o revelarla desde las notas) antes de que comiencen a programar. Enfatizar la regla: la primera pulsación da el dígito, las siguientes dan las letras, por lo que la n -ésima letra requiere $n + 1$ pulsaciones.

Implementación (35 min):

Dar libertad para que cada quien diseñe su solución. Los dos enfoques más comunes que surgirán:

1. **Cascada de `if / else` o `switch`**: un caso por letra. Funciona pero produce ~100 líneas. No disuadir; después mostrar la versión con arreglos.
2. **Dos arreglos `tecla[]` y `posicion[]`**: la solución de referencia. Señalar cómo `letra - 'a'` da el índice de 0 a 25, conectando con lo visto en la sesión de ASCII.

Circular por el salón. Los puntos de fricción más comunes son:

- Inicializar los dos arreglos correctamente (es tedioso; normal).
- El ciclo interno de pulsaciones: `for(int k=0; k<=pos; k++)`.
- Imprimir el espacio entre palabras pero no al final: `if(i > 1) cout << " ";` antes de cada palabra.

Discusión (15 min):

Pedir a dos o tres alumnos que proyecten su solución. Elegir una que use cascada y otra que use arreglos para comparar. Preguntar al grupo: “¿cuál prefieren? ¿Por qué?” Conectar con el principio de separar datos de lógica.

¿Por qué $n + 1$ pulsaciones y no n ?

En la mayoría de los teléfonos analógicos la primera pulsación de una tecla insertaba el dígito (el 4, no la g). Las letras se obtenían con pulsaciones adicionales. Esa convención explica que la primera letra requiera 2 pulsaciones.

¿Qué pasa con dos letras consecutivas en la misma tecla, como “gh” o “mn”?

En los teléfonos reales había que esperar un segundo (tiempo de espera) o presionar # para indicar que la siguiente pulsación era una letra nueva. Para esta actividad el enunciado garantiza que no habrá esa situación, así que no es necesario manejarla. Es un buen ejercicio adicional si hay tiempo: ¿cómo detectarías que la siguiente letra está en la misma tecla?

¿Por qué no usar un `map<char, string>`?

`std::map` no se ha visto aún. La solución con arreglos es deliberadamente más básica y refuerza la aritmética de `char`. Con `map` sería válida si los alumnos la conocen de otro contexto.

Soluciones a los ejercicios

```
#include <iostream>
using namespace std;

int main(int argc, char* argv[]){
    for(int i = 1; i < argc; i++)
        cout << i << ": " << argv[i] << endl;
}
```

Si no hay argumentos (`argc == 1`), el ciclo no itera y no se imprime nada. No se necesita verificación especial.

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main(int argc, char* argv[]){
    if(argc < 2){
        cerr << "Uso: " << argv[0] << " <num1> [num2 ...]" << endl;
        return 1;
    }
    int max = atoi(argv[1]);
    for(int i = 2; i < argc; i++){
        int n = atoi(argv[i]);
        if(n > max) max = n;
    }
    cout << max << endl;
}
```

Nótese que el máximo se inicializa con `argv[1]` (el primer argumento) y el ciclo empieza en 2. Inicializarlo con 0 o con `INT_MIN` también funciona pero depende de suposiciones sobre el rango de los datos.

```
#include <iostream>
using namespace std;

int main(int argc, char* argv[]){
    for(int i = argc-1; i >= 1; i--)
        cout << argv[i] << endl;
}
```

El ciclo va de `argc-1` hasta 1 (no hasta 0, para no imprimir el nombre del

programa). Si no hay argumentos, `argc-1 == 0 < 1` y el ciclo no itera.

```
#include <iostream>
using namespace std;

int main(int argc, char* argv[]){
    if(argc < 3){
        cerr << "Uso: " << argv[0]
            << " <letra> <palabra1> [palabra2 ...]" << endl;
        return 1;
    }
    char letra = argv[1][0];
    int cuenta = 0;
    for(int i = 2; i < argc; i++){
        for(int j = 0; argv[i][j] != '\0'; j++){
            if(argv[i][j] == letra) cuenta++;
        }
    }
    cout << cuenta << endl;
}
```

Verificación: ./contador a banana manzana naranja

- "banana": a en posiciones 1,3,5 → 3 aces.
- "manzana": a en posiciones 1,4,6 → 3 aces.
- "naranja": a en posiciones 1,3,5 → 3 aces...

Espera, contemos bien: "banana" = b-a-n-a-n-a → 3 a's. "manzana" = m-a-n-z-a-n-a → 3 a's. "naranja" = n-a-r-a-n-j-a → 3 a's. Total: **9**, no 7. El enunciado del ejercicio tiene la respuesta 7 como ejemplo ilustrativo; corregirlo en clase si algún alumno lo señala. Buen momento para enfatizar la importancia de verificar los ejemplos a mano.

```
#include <iostream>
using namespace std;

// Tecla y posicion de cada letra del alfabeto (indice: 'a'=0 ... 'z'=25)
int tecla[] = {2,2,2, 3,3,3, 4,4,4, 5,5,5, 6,6,6, 7,7,7,7, 8,8,8, 9,9,9,9};
int posicion[] = {1,2,3, 1,2,3, 1,2,3, 1,2,3, 1,2,3, 1,2,3,4, 1,2,3, 1,2,3,4};

int main(int argc, char* argv[]){
    if(argc < 2){
```

```

    cerr << "Uso: " << argv[0] << " <palabra1> [palabra2 ...]" << endl;
    return 1;
}
for(int i = 1; i < argc; i++){
    if(i > 1) cout << " ";
    for(int j = 0; argv[i][j] != '\0'; j++){
        int idx = argv[i][j] - 'a';
        for(int k = 0; k <= posicion[idx]; k++){
            cout << tecla[idx];
        }
    }
    cout << endl;
}

```

Verificación manual de hola:

Letra	Tecla	Pos	Pulsaciones (k <= pos)	Salida
h (idx=7)	4	2	k=0,1,2 → 3 veces	444
o (idx=14)	6	3	k=0,1,2,3 → 4 veces	6666
l (idx=11)	5	3	k=0,1,2,3 → 4 veces	5555
a (idx=0)	2	1	k=0,1 → 2 veces	22

Salida: 4446666555522 ✓

Nota sobre los arreglos:

- `tecla[]` tiene 26 elementos: $3+3+3+3+3+4+3+4 = 26$.
- `posicion[]` ídem.
- `argv[i][j] != '\0'` itera sobre los caracteres de cada cadena hasta el terminador.
- El ciclo de pulsaciones `for(k=0; k<=posicion[idx]; k++)` ejecuta `posicion[idx]+1` veces. Para `posicion=1` (letras a, d, g...) ejecuta 2 veces; para `posicion=4` (s,z) ejecuta 5 veces.

Errores frecuentes

1. Acceder a `argv[i]` sin verificar `argc`.

Síntoma: *segmentation fault* al ejecutar sin argumentos.

Remedio: verificar `argc` antes de acceder a cualquier `argv[i]` con $i \geq 1$.

2. Usar `argv[i]` directamente en aritmética.

Síntoma: resultados incorrectos o el compilador rechaza la operación.

Ejemplo típico: `int n = argv[1] + 1;` (debería ser `atoi(argv[1]) + 1`).

Remedio: convertir siempre con `atoi`, `atof` o asignación a `string`.

3. Ciclo que empieza en 0 e incluye `argv[0]` .

Síntoma: el nombre del programa aparece en la salida o se intenta operar sobre él como si fuera un dato.

Remedio: los argumentos del usuario siempre empiezan en `argv[1]` . El ciclo debe comenzar en `i = 1` .

4. En `convertir.cpp`: arreglos `tecla[]` con 26 elementos contados mal.

Síntoma: letras de la u en adelante producen resultados erróneos o el programa se sale de los límites del arreglo.

Remedio: contar los tres bloques: $3+3+3+3+3=15$ letras en teclas 2–6, luego 4 en la 7, 3 en la 8 y 4 en la 9 = 26. Verificar con `sizeof(tecla)/sizeof(int)` .

5. El espacio entre palabras aparece al inicio o al final.

Síntoma: " hola luna" o "hola luna ".

Remedio: imprimir el espacio *antes* de cada palabra excepto la primera: `if(i > 1) cout << " ";` al inicio del ciclo exterior.

Rúbrica de la actividad

Criterio	Puntos	Evidencia
<i>Recepción de argumentos</i>		
Usa <code>argc</code> y <code>argv</code> correctamente	10	código
Verifica <code>argc</code> y muestra uso en <code>cerr</code>	5	prueba sin args
<i>Mapeo letra–pulsaciones</i>		
Produce la secuencia correcta para cada letra	15	tabla de pruebas
Teclas 7 (pqrs) y 9 (wxyz) con 4 letras correctas	10	palabras con s/z
<i>Formato de salida</i>		
Espacio entre palabras, sin espacio al inicio/final	10	salida con 2+ palabras
Total	50	

Conexiones con temas posteriores

- **Sesión 24 y posteriores — Entrada estándar vs. args:** La diferencia entre `cin` (lectura interactiva) y `argv` (configuración al invocar) es un patrón que aparece en todos los programas de línea de comandos. Herramientas como `g++`, `git` y `grep` combinan ambas.
- **Orientación a Objetos — Constructores con parámetros:** La idea de que una entidad recibe su configuración al momento de crearse (no después) es la misma en `main(argc, argv)` y en un constructor con argumentos. Vale la pena señalarlo cuando se introduzcan los constructores.
- **Programación de sistemas — `getopt` y `argparse`:** Para programas reales con múltiples opciones opcionales (`-v`, `-output` `archivo`) existe la biblioteca `getopt`

en C/C++ y `argparse` en Python. Ambas procesan `argv` internamente; lo que aprendieron hoy es la base sobre la que se construyen.

■ Materiales y recursos

- **Notas alumno:** `notas_alumno/pdf/sesion23_argumentos_consola.pdf`
- **Material inicial:** `materialInicial/sesion23/` — `actividad`, `readme`, `teclado_numerico.jpg`
- **Imagen del teclado:** proyectar durante la introducción de la actividad; está disponible en el material inicial.